

Java Database Connectivity

Java + MySQL Connection (JDBC) - Complete Notes

Theory + Explanation + Code + Viva Questions

1. What is JDBC? (Introduction)

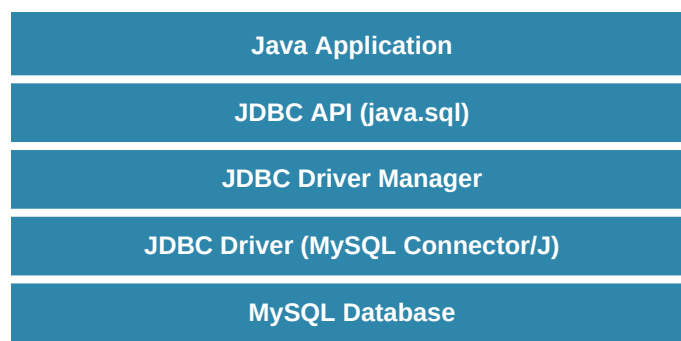
JDBC (Java Database Connectivity) is a Java API that allows a Java program to connect to a relational database (MySQL, Oracle, PostgreSQL, SQL Server, etc.) and **insert, update, delete and retrieve** data.

- JDBC lives in the package **java.sql** (and **javax.sql** for extended features).
- Java does not understand databases on its own, so every database has its own **JDBC Driver**. For MySQL it is **MySQL Connector/J**.
- JDBC provides a **standard set of interfaces** and the driver implements them. This means your code stays almost the same even if you change the database.

Simple analogy: Java program = Customer, JDBC Driver = Translator, MySQL = Shopkeeper. The translator (driver) carries the customer's request to the shopkeeper in the language the shopkeeper understands.

2. JDBC Architecture

JDBC architecture has the following layers:



Layer	Role
Java Application	Your program that sends SQL queries.
JDBC API	Provides interfaces and classes (Connection, Statement, ResultSet...).
DriverManager	Selects the correct driver and creates the connection.
JDBC Driver	Converts Java calls into the database-specific protocol.

Layer	Role
Database	Stores data, executes the query and returns the result.

3. Types of JDBC Drivers

Type	Name	Description
Type 1	JDBC-ODBC Bridge	Connects through ODBC. Removed in Java 8.
Type 2	Native-API Driver	Uses the database's native C/C++ library. Platform dependent.
Type 3	Network Protocol Driver	Connects through a middleware server.
Type 4	Thin Driver (Pure Java)	Talks directly to the database. Fastest and most popular. MySQL Connector/J is Type 4.

4. Setup Before Connecting (Prerequisites)

- **JDK** must be installed.
- **MySQL Server** must be installed and running (default port 3306).
- **MySQL Connector/J** (.jar file) must be added to the project classpath. If you use Maven, add the dependency instead.

Step A: Create the Database

```
CREATE DATABASE studentdb;
USE studentdb;
```

Step B: Maven Dependency (if using Maven)

```
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <version>8.4.0</version>
</dependency>
```

Step C: Adding the JAR manually (Eclipse / IntelliJ / Command line)

- **Eclipse:** Right-click project -> Build Path -> Add External JARs -> select the connector jar.
- **IntelliJ:** File -> Project Structure -> Libraries -> + -> Java -> select the jar.
- **Command line:** javac Test.java then java -cp .;mysql-connector-j.jar Test (use : instead of ; on Linux/Mac).

5. The 5 Steps of a JDBC Connection

Step	Task	Code
1	Import the packages	import java.sql.*;

Step	Task	Code
2	Load the driver (optional in modern JDBC)	<code>Class.forName("com.mysql.cj.jdbc.Driver");</code>
3	Establish the connection	<code>DriverManager.getConnection(url,user,pass);</code>
4	Create a statement and execute the query	<code>con.createStatement();</code> <code>stmt.executeQuery(sql);</code>
5	Close the connection	<code>con.close();</code>

Note: Since JDBC 4.0 (Java 6+), `Class.forName()` is not required because the driver is loaded automatically through *META-INF/services* inside the jar. However, this step is still frequently asked in exams.

6. Main Program (Connection Code)

```
import java.sql.*;

public class Test {

    public static void main(String[] args) {

        try {

            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/studentdb",
                "root",
                "1234"
            );

            System.out.println("Connected Successfully!");

        } catch (Exception e) {

            System.out.println(e);

        }

    }

}
```

Output:

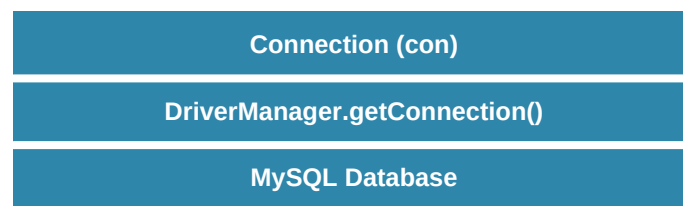
```
Connected Successfully!
```

Line-by-Line Explanation

Line	Explanation
<code>import java.sql.*;</code>	Imports all JDBC classes/interfaces (Connection, DriverManager, SQLException...).
<code>Connection con</code>	Connection is an interface that represents an active session with the database.

Line	Explanation
<code>DriverManager.getConnection(...)</code>	DriverManager is a class that finds a suitable registered driver and returns a connection object.
<code>"jdbc:mysql://..."</code>	The database URL (address).
<code>"root"</code>	The MySQL username.
<code>"1234"</code>	The MySQL password.
<code>catch (Exception e)</code>	Catches any error if the connection fails (wrong password, server down, driver missing). Catching SQLException specifically is better practice.

7. Connection Flow



The Java program gives the URL, username and password to **DriverManager.getConnection()**. DriverManager contacts the MySQL server through the driver. If the credentials are correct, a **Connection object** is returned; otherwise a **SQLException** is thrown.

8. Understanding the JDBC URL

<code>"jdbc:mysql://localhost:3306/studentdb"</code>	
Part	Meaning
jdbc	Protocol - indicates this is a JDBC connection.
mysql	Sub-protocol - which database is used (for Oracle it is <i>oracle:thin</i>).
localhost	Host - MySQL is running on your own computer. For a remote server, use its IP address or domain.
3306	Port number - the default MySQL port.
studentdb	Name of the database to connect to.

URL format: `jdbc:subprotocol://host:port/databaseName`

Username and password: root is MySQL's default admin user and 1234 is that user's password. Your own password may be different - use the one you set when installing MySQL.

9. Important JDBC Classes and Interfaces

Name	Type	Purpose
DriverManager	Class	Manages drivers and creates connections.
Connection	Interface	Session with the database. Used to create statements and manage transactions.
Statement	Interface	Executes simple SQL queries.
PreparedStatement	Interface	Precompiled query with parameters (?). Faster and safe from SQL injection .
CallableStatement	Interface	Used to call stored procedures.
ResultSet	Interface	Holds the result (rows) of a SELECT query.
SQLException	Class	Exception for database errors.

10. Best Practice: try-with-resources

Always close the connection, otherwise resources leak. From Java 7 onwards, **try-with-resources** closes it automatically:

```
import java.sql.*;

public class Test {
    public static void main(String[] args) {

        String url = "jdbc:mysql://localhost:3306/studentdb";
        String user = "root";
        String pass = "1234";

        try (Connection con = DriverManager.getConnection(url, user, pass)) {

            System.out.println("Connected Successfully!");

        } catch (SQLException e) {
            System.out.println("Connection Failed: " + e.getMessage());
        }
    }
}
```

11. Next Step: Create a Table and Fetch Data (Bonus)

```
CREATE TABLE student (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    marks INT
);
INSERT INTO student VALUES (1, 'Rahul', 85), (2, 'Priya', 92);
```

```
import java.sql.*;

public class FetchData {
    public static void main(String[] args) {
        try (Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/studentdb", "root", "1234");
            Statement stmt = con.createStatement();
            ResultSet rs = stmt.executeQuery("SELECT * FROM student")) {

            while (rs.next()) {
                System.out.println(rs.getInt("id") + " "
                    + rs.getString("name") + " "
                    + rs.getInt("marks"));
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Output:

```
1 Rahul 85
2 Priya 92
```

12. Common Errors and Solutions

Error	Reason	Solution
ClassNotFoundException <i>/ No suitable driver found</i>	Connector/J jar is not on the classpath.	Add the jar to the Build Path / classpath.
Access denied for user 'root'@'localhost'	Wrong username or password.	Use the correct MySQL password.
Unknown database 'studentdb'	The database has not been created.	Run <code>CREATE DATABASE studentdb;</code>
Communications link failure	MySQL server is stopped or the port is wrong.	Start the MySQL service and check port 3306.
Public Key Retrieval is not allowed	MySQL 8 authentication issue.	Append <code>?allowPublicKeyRetrieval=true&useSSL=false</code> to the URL.
Timezone error	Seen with older connector versions.	Append <code>?serverTimezone=UTC</code> to the URL.

13. Important Viva / Exam Questions

Q1. What is JDBC?

Ans: A Java API that lets a Java program connect to a relational database.

Q2. What is a JDBC driver?

Ans: A software component that translates Java calls into database-specific calls.

Q3. Why is a Type 4 driver called a thin driver?

Ans: It is written entirely in Java and talks directly to the database without any native library.

Q4. What does DriverManager do?

Ans: It manages the registered drivers and establishes the connection.

Q5. Difference between Connection, Statement and ResultSet?

Ans: Connection = session with the DB, Statement = object used to send queries, ResultSet = the result of a query.

Q6. Statement vs PreparedStatement?

Ans: PreparedStatement is precompiled, faster, and protects against SQL injection.

Q7. What is the default MySQL port?

Ans: 3306.

Q8. Why must a connection be closed?

Ans: To free database resources and avoid memory/connection leaks.

Q9. What does Class.forName() do?

Ans: It loads the driver class into memory and registers it with DriverManager (optional since JDBC 4).

14. Quick Revision

Remember these 3 things:

1. **Connection** - the object representing the session with the database.
2. **DriverManager.getConnection(url, user, password)** - creates the connection.
3. **MySQL Database** - where the data is stored.

URL: jdbc:mysql://localhost:3306/studentdb

localhost = your computer | 3306 = MySQL port | studentdb = database | root = username | 1234 = password

Flow: Java Program -> Connection -> DriverManager.getConnection() -> JDBC Driver -> MySQL Database